

Solution to Homework 3

Meshach Aruoriwo Ogbor,
Md Rasel

November 1, 2023

1 Exercise 3.1 - Point to Point Communication

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <mpi.h>
4
5 // custom function my_Bcast_Loop
6 int my_Bcast_Loop(void *buffer, int count,
7 MPI_Datatype datatype, int root, MPI_Comm comm) {
8     int numprocs, rank;
9
10    MPI_Comm_size(comm, &numprocs);
11    MPI_Comm_rank(comm, &rank);
12
13    if(rank == root) {
14        for(int i = 0; i < numprocs; i++) {
15            if(i != root) {
16                MPI_Send(buffer, count, datatype, i,
17                        0, comm);
18            }
19        }
20    } else {
21        MPI_Recv(buffer, count, datatype, root, 0,
22                comm, MPI_STATUS_IGNORE);
23    }
24
25    return MPI_SUCCESS;
26 }
```

```

24
25 // main function
26 int main(int argc, char* argv[]) {
27
28     int root = 0;
29     int numprocs, rank;
30
31     MPI_Init(&argc, &argv);
32
33     MPI_Comm_rank(MPLCOMM_WORLD, &rank);
34     MPI_Comm_size(MPLCOMM_WORLD, &numprocs);
35
36     // allocating memory for an array of 100000
37     doubles
38     double* data = (double*)malloc(sizeof(double)
39         *100000);
40     double* data_copy = (double*)malloc(sizeof(double)
41         *100000);
42
43     if(rank == root) {
44         for(int i = 0; i < 100000; i++) {
45             data[i] = i;
46             data_copy[i] = i;
47         }
48     }
49
50     // measuring and printing the time taken by
51     MPI_Bcast
52     MPI_Barrier(MPLCOMM_WORLD);
53     double start = MPI_Wtime();
54     MPI_Bcast(data, 100000, MPLDOUBLE, root,
55         MPLCOMM_WORLD);
56     MPI_Barrier(MPLCOMM_WORLD);
57     double end = MPI_Wtime();
58     printf("MPI_Bcast time(seconds) for rank %2d: %f\n",
59         end-start, rank);
60
61     // measuring and printing the time taken by
62     MPI_Bcast_Loop

```

```

57     MPI_Barrier(MPLCOMM_WORLD);
58     start = MPI_Wtime();
59     my_Bcast_Loop(data_copy, 100000, MPI_DOUBLE, root,
60                   MPLCOMM_WORLD);
61     MPI_Barrier(MPLCOMM_WORLD);
62     end = MPI_Wtime();
63     printf("my_Bcast_Loop time(seconds) for rank %2d:
64           %f\n", end-start, rank);
65
66     // for verification
67     for (int i = 0; i < 100000; i++){
68         if(data[i] != data_copy[i]) {
69             printf("Mismatch at index %2d: %f
70                   vs %f\n", i, data[i], data_copy
71                   [i]);
72             break;
73         }
74     }
75     free(data);
76     free(data_copy);
77     MPI_Finalize();
78     return 0;
79 }

```

For compilation on Stromboli, we used the following command:
`mpicc -o ex3_1 ex3_1.c`

2 Submit Script for exercise 3.1

```
1 #!/bin/bash
2 #SBATCH --nodes=3
3 #SBATCH --ntasks=48
4 #SBATCH --partition=compute2011
5 #SBATCH --exclusive
6 #SBATCH --output=output.txt
7
8 module load mpi/openmpi/4.1.0
9 mpirun ./ex3_1
```

3 Results

```
MPI_Bcast time(seconds) for rank 0: 0.014150
MPI_Bcast time(seconds) for rank 1: 0.014103
MPI_Bcast time(seconds) for rank 2: 0.014189
MPI_Bcast time(seconds) for rank 3: 0.014051
MPI_Bcast time(seconds) for rank 4: 0.014167
MPI_Bcast time(seconds) for rank 5: 0.014153
MPI_Bcast time(seconds) for rank 6: 0.014728
MPI_Bcast time(seconds) for rank 7: 0.014581
MPI_Bcast time(seconds) for rank 8: 0.014839
MPI_Bcast time(seconds) for rank 9: 0.014701
MPI_Bcast time(seconds) for rank 10: 0.013400
MPI_Bcast time(seconds) for rank 11: 0.014646
MPI_Bcast time(seconds) for rank 12: 0.017116
MPI_Bcast time(seconds) for rank 13: 0.017105
MPI_Bcast time(seconds) for rank 14: 0.016926
MPI_Bcast time(seconds) for rank 15: 0.016930
MPI_Bcast time(seconds) for rank 31: 0.024325
MPI_Bcast time(seconds) for rank 34: 0.014172
MPI_Bcast time(seconds) for rank 16: 0.024453
MPI_Bcast time(seconds) for rank 35: 0.014041
MPI_Bcast time(seconds) for rank 17: 0.024379
MPI_Bcast time(seconds) for rank 36: 0.014140
MPI_Bcast time(seconds) for rank 18: 0.024409
MPI_Bcast time(seconds) for rank 37: 0.014122
MPI_Bcast time(seconds) for rank 19: 0.024409
MPI_Bcast time(seconds) for rank 38: 0.014740
```

```

MPI_Bcast time(seconds) for rank 20: 0.024434
MPI_Bcast time(seconds) for rank 39: 0.014567
MPI_Bcast time(seconds) for rank 21: 0.024328
MPI_Bcast time(seconds) for rank 40: 0.014833
MPI_Bcast time(seconds) for rank 22: 0.024429
MPI_Bcast time(seconds) for rank 41: 0.014700
MPI_Bcast time(seconds) for rank 23: 0.024341
MPI_Bcast time(seconds) for rank 42: 0.013385
MPI_Bcast time(seconds) for rank 24: 0.024394
MPI_Bcast time(seconds) for rank 43: 0.014634
MPI_Bcast time(seconds) for rank 25: 0.024359
MPI_Bcast time(seconds) for rank 44: 0.017098
MPI_Bcast time(seconds) for rank 26: 0.024426
MPI_Bcast time(seconds) for rank 45: 0.017082
MPI_Bcast time(seconds) for rank 27: 0.024359
MPI_Bcast time(seconds) for rank 46: 0.016930
MPI_Bcast time(seconds) for rank 28: 0.024347
MPI_Bcast time(seconds) for rank 47: 0.016926
MPI_Bcast time(seconds) for rank 29: 0.024367
MPI_Bcast time(seconds) for rank 32: 0.014119
MPI_Bcast time(seconds) for rank 30: 0.024409
MPI_Bcast time(seconds) for rank 33: 0.014079
my_Bcast_Loop time(seconds) for rank 8: 0.123684
my_Bcast_Loop time(seconds) for rank 9: 0.123661
my_Bcast_Loop time(seconds) for rank 10: 0.123661
my_Bcast_Loop time(seconds) for rank 11: 0.123672
my_Bcast_Loop time(seconds) for rank 12: 0.123680
my_Bcast_Loop time(seconds) for rank 13: 0.123681
my_Bcast_Loop time(seconds) for rank 14: 0.123659
my_Bcast_Loop time(seconds) for rank 15: 0.123660
my_Bcast_Loop time(seconds) for rank 0: 0.123695
my_Bcast_Loop time(seconds) for rank 1: 0.123672
my_Bcast_Loop time(seconds) for rank 2: 0.123694
my_Bcast_Loop time(seconds) for rank 3: 0.123704
my_Bcast_Loop time(seconds) for rank 4: 0.123688
my_Bcast_Loop time(seconds) for rank 5: 0.123716
my_Bcast_Loop time(seconds) for rank 6: 0.123688
my_Bcast_Loop time(seconds) for rank 7: 0.123668
my_Bcast_Loop time(seconds) for rank 31: 0.123666
my_Bcast_Loop time(seconds) for rank 39: 0.123666

```

my_Bcast_Loop time(seconds) for rank 16: 0.123695
my_Bcast_Loop time(seconds) for rank 40: 0.123683
my_Bcast_Loop time(seconds) for rank 17: 0.123661
my_Bcast_Loop time(seconds) for rank 41: 0.123660
my_Bcast_Loop time(seconds) for rank 18: 0.123661
my_Bcast_Loop time(seconds) for rank 42: 0.123661
my_Bcast_Loop time(seconds) for rank 19: 0.123677
my_Bcast_Loop time(seconds) for rank 43: 0.123670
my_Bcast_Loop time(seconds) for rank 20: 0.123680
my_Bcast_Loop time(seconds) for rank 44: 0.123678
my_Bcast_Loop time(seconds) for rank 21: 0.123701
my_Bcast_Loop time(seconds) for rank 45: 0.123676
my_Bcast_Loop time(seconds) for rank 22: 0.123673
my_Bcast_Loop time(seconds) for rank 46: 0.123659
my_Bcast_Loop time(seconds) for rank 23: 0.123658
my_Bcast_Loop time(seconds) for rank 33: 0.123670
my_Bcast_Loop time(seconds) for rank 24: 0.123688
my_Bcast_Loop time(seconds) for rank 47: 0.123660
my_Bcast_Loop time(seconds) for rank 25: 0.123649
my_Bcast_Loop time(seconds) for rank 32: 0.123695
my_Bcast_Loop time(seconds) for rank 26: 0.123655
my_Bcast_Loop time(seconds) for rank 34: 0.123691
my_Bcast_Loop time(seconds) for rank 27: 0.123663
my_Bcast_Loop time(seconds) for rank 35: 0.123701
my_Bcast_Loop time(seconds) for rank 36: 0.123685
my_Bcast_Loop time(seconds) for rank 28: 0.123689
my_Bcast_Loop time(seconds) for rank 29: 0.123680
my_Bcast_Loop time(seconds) for rank 37: 0.123714
my_Bcast_Loop time(seconds) for rank 30: 0.123652
my_Bcast_Loop time(seconds) for rank 38: 0.123685

4 Exercise 3.2 - Tree

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <mpi.h>
4 #include <math.h>
5
6 // custom function to broadcast data in a tree-like
  fashion
7 int my_Bcast_Tree(void *buffer, int count,
  MPI_Datatype datatype, int root, MPI_Comm comm) {
8     int numprocs, rank;
9
10    MPI_Comm_size(comm, &numprocs);
11    MPI_Comm_rank(comm, &rank);
12
13    // finding smallest integer k with numprocs <= 2^k
14    int k = ceil(log2(numprocs));
15
16    // loop from k down to 1
17    for(int i = k; i >= 1; i--) {
18        // all processes with rank mod 2^i = root:
19        // send data to process with rank rank+2^(i-1)
20        // (if it exists)
21        if(rank % (int)pow(2, i) == root) {
22            int dest = rank + pow(2, i-1);
23            if(dest < numprocs) {
24                MPI_Send(buffer, count, datatype, dest
25                    , 0, comm);
26            }
27        }
28        // processes with rank mod 2^(i-1) = root:
29        // receive data from process with rank rank
30        // -2^(i-1)
31        else if(rank % (int)pow(2, i-1) == root) {
32            MPI_Recv(buffer, count, datatype, rank -
33                pow(2, i-1), 0, comm, MPI_STATUS_IGNORE
34                );
35        }
36    }
```

```

29     }
30
31     return MPI_SUCCESS;
32 }
33
34 // main function
35 int main(int argc, char* argv[]) {
36     int root = 0;
37     int numprocs, rank;
38
39     MPI_Init(&argc, &argv);
40
41     MPI_Comm_size(MPLCOMM_WORLD, &numprocs);
42     MPI_Comm_rank(MPLCOMM_WORLD, &rank);
43
44     // allocating memory for an array of 100000
45     doubles
46     double* data = (double*)malloc(sizeof(double)
47         *100000);
48     double* data_copy = (double*)malloc(sizeof(double)
49         *100000);
50
51     // if this is the root process, initialize the
52     data array
53     if(rank == root) {
54         for(int i = 0; i < 100000; i++) {
55             data[i] = i;
56             data_copy[i] = i;
57         }
58     }
59
60     // measure and print the time taken by MPI_Bcast
61     double start = MPI_Wtime();
62     MPI_Bcast(data, 100000, MPLDOUBLE, root,
63         MPLCOMM_WORLD);
64     double end = MPI_Wtime();
65     printf("MPI_Bcast time(seconds) for rank %2d: %f\n",
66         end-start, rank);
67
68     // measure and print the time taken by

```

```

        my_Bcast_Tree
63     MPI_Barrier(MPLCOMM_WORLD);
64     start = MPI_Wtime();
65     my_Bcast_Tree(data_copy, 100000, MPLDOUBLE, root,
        MPLCOMM_WORLD);
66     MPI_Barrier(MPLCOMM_WORLD);
67     end = MPI_Wtime();
68     printf("my_Bcast_Tree time(seconds) for rank %2d:%f\n", end-start, rank);
69
70     // for verification
71     for (int i = 0; i < 100000; i++){
72         if(data[i] != data_copy[i]) {
73             printf("Mismatch at index %2d: %f
        vs %f\n", i, data[i], data_copy
        [i]);
74             break;
75         }
76     }
77
78     free(data);
79     free(data_copy);
80
81     MPI_Finalize();
82     return 0;
83 }

```

For compilation on Stromboli, we used the following command:

```
mpicc -o ex3_2 ex3_2.c -lm
```

5 Submit Script for exercise 3.2

```
1 #!/bin/bash
2 #SBATCH --nodes=3
3 #SBATCH --ntasks=48
4 #SBATCH --partition=compute2011
5 #SBATCH --exclusive
6 #SBATCH --output=output3_2.txt
7
8 module load mpi/openmpi/4.1.0
9 mpirun ./ex3_2
```

6 Results

```
MPI_Bcast time(seconds) for rank 2: 0.014563
MPI_Bcast time(seconds) for rank 3: 0.014528
MPI_Bcast time(seconds) for rank 1: 0.014549
MPI_Bcast time(seconds) for rank 9: 0.015954
MPI_Bcast time(seconds) for rank 11: 0.016806
MPI_Bcast time(seconds) for rank 12: 0.017620
MPI_Bcast time(seconds) for rank 13: 0.017559
MPI_Bcast time(seconds) for rank 14: 0.017656
MPI_Bcast time(seconds) for rank 15: 0.017724
MPI_Bcast time(seconds) for rank 4: 0.017851
MPI_Bcast time(seconds) for rank 6: 0.017530
MPI_Bcast time(seconds) for rank 7: 0.017677
MPI_Bcast time(seconds) for rank 5: 0.017757
MPI_Bcast time(seconds) for rank 0: 0.020019
MPI_Bcast time(seconds) for rank 35: 0.022600
MPI_Bcast time(seconds) for rank 32: 0.022709
MPI_Bcast time(seconds) for rank 34: 0.022635
MPI_Bcast time(seconds) for rank 33: 0.022608
MPI_Bcast time(seconds) for rank 8: 0.023293
MPI_Bcast time(seconds) for rank 10: 0.023292
MPI_Bcast time(seconds) for rank 19: 0.023477
MPI_Bcast time(seconds) for rank 17: 0.008741
MPI_Bcast time(seconds) for rank 42: 0.018063
MPI_Bcast time(seconds) for rank 43: 0.021880
MPI_Bcast time(seconds) for rank 40: 0.025538
MPI_Bcast time(seconds) for rank 41: 0.017569
```

MPI_Bcast time(seconds) for rank 45: 0.025781
MPI_Bcast time(seconds) for rank 47: 0.025830
MPI_Bcast time(seconds) for rank 36: 0.025882
MPI_Bcast time(seconds) for rank 37: 0.025746
MPI_Bcast time(seconds) for rank 38: 0.025785
MPI_Bcast time(seconds) for rank 39: 0.019247
MPI_Bcast time(seconds) for rank 29: 0.027219
MPI_Bcast time(seconds) for rank 30: 0.027134
MPI_Bcast time(seconds) for rank 31: 0.027582
MPI_Bcast time(seconds) for rank 27: 0.007619
MPI_Bcast time(seconds) for rank 24: 0.007076
MPI_Bcast time(seconds) for rank 25: 0.004738
MPI_Bcast time(seconds) for rank 26: 0.011773
MPI_Bcast time(seconds) for rank 18: 0.012724
MPI_Bcast time(seconds) for rank 16: 0.028338
MPI_Bcast time(seconds) for rank 28: 0.029719
MPI_Bcast time(seconds) for rank 21: 0.023582
MPI_Bcast time(seconds) for rank 22: 0.012067
MPI_Bcast time(seconds) for rank 44: 0.032139
MPI_Bcast time(seconds) for rank 46: 0.032135
MPI_Bcast time(seconds) for rank 20: 0.039300
MPI_Bcast time(seconds) for rank 23: 0.021113
my_Bcast_Tree time(seconds) for rank 2:0.008867
my_Bcast_Tree time(seconds) for rank 15:0.011118
my_Bcast_Tree time(seconds) for rank 7:0.017251
my_Bcast_Tree time(seconds) for rank 4:0.008793
my_Bcast_Tree time(seconds) for rank 6:0.014413
my_Bcast_Tree time(seconds) for rank 0:0.032437
my_Bcast_Tree time(seconds) for rank 1:0.004042
my_Bcast_Tree time(seconds) for rank 8:0.022495
my_Bcast_Tree time(seconds) for rank 9:0.020294
my_Bcast_Tree time(seconds) for rank 11:0.018030
my_Bcast_Tree time(seconds) for rank 12:0.010395
my_Bcast_Tree time(seconds) for rank 13:0.010432
my_Bcast_Tree time(seconds) for rank 14:0.007797
my_Bcast_Tree time(seconds) for rank 10:0.015941
my_Bcast_Tree time(seconds) for rank 5:0.009057
my_Bcast_Tree time(seconds) for rank 3:0.010866
my_Bcast_Tree time(seconds) for rank 40:0.022501
my_Bcast_Tree time(seconds) for rank 18:0.009072

```

my_Bcast_Tree time(seconds) for rank 33:0.004049
my_Bcast_Tree time(seconds) for rank 19:0.011078
my_Bcast_Tree time(seconds) for rank 42:0.015949
my_Bcast_Tree time(seconds) for rank 26:0.020572
my_Bcast_Tree time(seconds) for rank 39:0.017242
my_Bcast_Tree time(seconds) for rank 22:0.018367
my_Bcast_Tree time(seconds) for rank 41:0.020283
my_Bcast_Tree time(seconds) for rank 28:0.016519
my_Bcast_Tree time(seconds) for rank 44:0.010393
my_Bcast_Tree time(seconds) for rank 21:0.013451
my_Bcast_Tree time(seconds) for rank 45:0.010420
my_Bcast_Tree time(seconds) for rank 20:0.013040
my_Bcast_Tree time(seconds) for rank 46:0.007805
my_Bcast_Tree time(seconds) for rank 27:0.032339
my_Bcast_Tree time(seconds) for rank 35:0.010865
my_Bcast_Tree time(seconds) for rank 24:0.031215
my_Bcast_Tree time(seconds) for rank 34:0.008885
my_Bcast_Tree time(seconds) for rank 25:0.024502
my_Bcast_Tree time(seconds) for rank 32:0.032439
my_Bcast_Tree time(seconds) for rank 31:0.016743
my_Bcast_Tree time(seconds) for rank 47:0.011109
my_Bcast_Tree time(seconds) for rank 16:0.032469
my_Bcast_Tree time(seconds) for rank 36:0.008766
my_Bcast_Tree time(seconds) for rank 17:0.005101
my_Bcast_Tree time(seconds) for rank 37:0.009061
my_Bcast_Tree time(seconds) for rank 29:0.016189
my_Bcast_Tree time(seconds) for rank 38:0.014401
my_Bcast_Tree time(seconds) for rank 30:0.013430
my_Bcast_Tree time(seconds) for rank 43:0.018056
my_Bcast_Tree time(seconds) for rank 23:0.030288

```

As you can see from the results, time taken by the custom “Tree” function is less than the custom “Loop” function. For example: on rank 0, it took around 4 times less time ($0.123695/0.032437$) while using the “Tree” function. Theoretically, we should get $\log_2 48 = 5.5$ times less time.